

Synthèse des modèles séquentiels et de la traduction automatique

Fiche pédagogique structurée en français

Cette synthèse fusionne en un seul document les notions essentielles sur les **modèles de langage**, les **RNN**, la **rétropropagation à travers le temps**, les **LSTM**, les **GRU**, les variantes **profondes et bidirectionnelles**, la préparation de données pour la **traduction automatique**, l'architecture **encodeur-décodeur**, les modèles **Seq2Seq** et le **beam search**. L'objectif est d'offrir une progression claire : *modéliser une séquence, l'entraîner, la traduire, puis la décoder efficacement*.

Version de synthèse unifiée

8 avril 2026

Table des matières

1	Vue d'ensemble	2
2	Modèles de langage	2
2.1	Objectif probabiliste	2
2.2	Approches n-grammes	2
2.3	Perplexité	3
3	Réseaux neuronaux récurrents (RNN)	3
3.1	Idée centrale	3
3.2	Interprétation matricielle condensée	3
3.3	Pourquoi un RNN ?	3
3.4	Exemple simple	4
3.5	Code minimal PyTorch	4
4	Apprentissage des RNN : BPTT	4
4.1	Dépliage temporel	4
4.2	Forme du gradient	4
4.3	Gradient clipping	5
4.4	BPTT tronquée	5
5	LSTM : Long Short-Term Memory	5
5.1	Pourquoi les LSTM ?	5
5.2	Équations	5
5.3	Rôle des portes	5
5.4	Code concis	6
6	GRU : Gated Recurrent Unit	6
6.1	Principe	6
6.2	Interprétation	6
6.3	Comparaison LSTM / GRU	6
7	Variantes architecturales : RNN profonds et bidirectionnels	6
7.1	RNN profonds	7
7.2	Dropout récurrent	7
7.3	RNN bidirectionnels	7
8	Préparation des données pour la traduction automatique	7
8.1	Pipeline général	7

8.2	Tokens spéciaux	7
8.3	Padding et masquage	8
8.4	Tenseurs et mini-lots	8
9	Architecture encodeur–décodeur	8
9.1	Idée générale	8
9.2	Formalisation	8
9.3	Interfaces conceptuelles	9
10	Seq2Seq récurrent	9
10.1	Encodeur GRU	9
10.2	Décodeur conditionnel	9
10.3	Teacher forcing	9
10.4	Fonction de perte	9
10.5	Inférence auto-régressive	9
10.6	Évaluation par BLEU	10
11	Décodage : glouton, exhaustif et beam search	10
11.1	Pourquoi le décodage est difficile ?	10
11.2	Décodage glouton	10
11.3	Score d’une séquence	10
11.4	Beam search	10
11.5	Pénalisation de longueur	11
12	Comparaison globale des modèles	11
13	Mini-résumé opérationnel	11
14	Exemple de squelette d’entraînement Seq2Seq en PyTorch	12
15	Conclusion	12

1 Vue d'ensemble

Les modèles séquentiels traitent des données ordonnées : texte, parole, séries temporelles, logs, trajectoires, ADN, etc. La difficulté centrale est de capturer la **dépendance entre positions** : le sens d'un mot ou la valeur future d'un signal dépend souvent de ce qui précède, parfois aussi de ce qui suit.

Une chaîne de traitement typique est la suivante :

- 1) représenter une séquence sous forme de tokens ou vecteurs ;
- 2) définir un modèle capable de mémoriser le contexte ;
- 3) entraîner le modèle par optimisation d'une perte ;
- 4) utiliser le modèle pour prédire, générer ou traduire ;
- 5) décoder intelligemment la sortie produite.

Séquence. Une séquence est une suite ordonnée $(x_1, x_2, \dots, x_T)$. La longueur T peut varier d'un exemple à l'autre. Le modèle doit donc être capable de gérer des entrées de longueur variable.

2 Modèles de langage

2.1 Objectif probabiliste

Un modèle de langage attribue une probabilité à une phrase ou à une suite de tokens :

$$P(x_1, x_2, \dots, x_T).$$

Par la règle de chaîne,

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

L'idée fondatrice est simple : pour modéliser une phrase entière, il suffit de savoir prédire **le prochain token à partir du passé**. Toute la difficulté est dans la représentation de ce passé.

2.2 Approches n-grammes

Dans les modèles de Markov d'ordre $n - 1$, on approxime :

$$P(x_t \mid x_1, \dots, x_{t-1}) \approx P(x_t \mid x_{t-n+1}, \dots, x_{t-1}).$$

Exemple bigramme :

$$P(x_1, \dots, x_T) \approx P(x_1) \prod_{t=2}^T P(x_t \mid x_{t-1}).$$

Avantages : simplicité, interprétabilité, calcul rapide.

Limites : explosion combinatoire, sparsité, incapacité à capturer de longues dépendances.

2.3 Perplexité

La performance d'un modèle de langage est souvent mesurée par la perplexité :

$$\text{PPL} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log P(x_t | x_{<t}) \right).$$

Plus la perplexité est faible, meilleur est le modèle.

Si le modèle hésite fortement sur chaque mot, la perplexité augmente. S'il prédit systématiquement le bon mot avec une forte probabilité, la perplexité diminue.

3 Réseaux neuronaux récurrents (RNN)

3.1 Idée centrale

Contrairement à un MLP classique, un RNN possède un **état caché** qui se transmet de pas en pas. À l'instant t , il combine l'entrée courante x_t avec la mémoire précédente h_{t-1} .

$$\begin{aligned} h_t &= \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \\ o_t &= W_{hq}h_t + b_q, \quad \hat{y}_t = \text{softmax}(o_t). \end{aligned}$$

Ici :

- x_t : entrée au temps t ;
- h_t : état caché ;
- o_t : logits de sortie ;
- ϕ : non-linéarité, souvent tanh.

État caché. C'est la mémoire interne du RNN. Il résume, sous forme vectorielle, ce qui a été observé jusque-là dans la séquence.

3.2 Interprétation matricielle condensée

On peut concaténer l'entrée et l'état précédent :

$$\tilde{x}_t = \begin{bmatrix} x_t \\ h_{t-1} \end{bmatrix}, \quad h_t = \phi(W\tilde{x}_t + b).$$

Cette écriture montre que le RNN est une transformation non linéaire appliquée à la fois à l'entrée présente et à la mémoire passée.

3.3 Pourquoi un RNN ?

Un RNN est adapté quand :

- la longueur des séquences varie ;
- l'ordre des éléments importe ;
- il faut exploiter un contexte accumulé.

3.4 Exemple simple

Pour le texte « je suis all... », le modèle doit prédire un mot plausible comme « é » ou «ée » au niveau caractère, ou « allé » au niveau mot. Cette prédiction dépend du contexte antérieur, d'où l'intérêt d'une mémoire récurrente.

3.5 Code minimal PyTorch

```
import torch
from torch import nn

rnn = nn.RNN(input_size=32, hidden_size=64, batch_first=True)
X = torch.randn(16, 10, 32)  # batch=16, longueur=10, dimension=32
H0 = torch.zeros(1, 16, 64)  # 1 couche, batch=16, hidden=64
Y, Hn = rnn(X, H0)
print(Y.shape)  # (16, 10, 64)
print(Hn.shape) # (1, 16, 64)
```

4 Apprentissage des RNN : BPTT

4.1 Dépliage temporel

Pour entraîner un RNN, on le **déplie dans le temps**. Un seul bloc récurrent partagé devient une chaîne de blocs identiques, un par instant.

La perte totale sur une séquence est souvent :

$$\mathcal{L} = \sum_{t=1}^T \ell_t.$$

Comme h_t dépend de h_{t-1} , le gradient par rapport aux paramètres doit traverser tous les pas temporels antérieurs. C'est la **Backpropagation Through Time** (BPTT).

4.2 Forme du gradient

Le gradient d'une perte finale par rapport à un état antérieur contient une chaîne de produits jacobiens :

$$\frac{\partial \mathcal{L}}{\partial h_k} = \sum_{t \geq k} \frac{\partial \mathcal{L}}{\partial h_t} \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}.$$

Cette expression explique deux phénomènes classiques :

- **gradient évanescent** : le produit tend vers zéro ;
- **gradient explosif** : le produit devient très grand.

Les RNN simples ont donc du mal à apprendre des dépendances très longues.

4.3 Gradient clipping

Pour limiter l'explosion, on utilise souvent le **clipping** : si la norme du gradient dépasse un seuil θ , on le renormalise.

$$g \leftarrow \min\left(1, \frac{\theta}{\|g\|}\right) g.$$

4.4 BPTT tronquée

Au lieu de rétropropager sur toute la séquence, on coupe l'horizon temporel après τ pas. Cela réduit le coût mémoire et le coût de calcul, au prix d'une vision plus courte du passé.

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
optimizer.zero_grad()
```

5 LSTM : Long Short-Term Memory

5.1 Pourquoi les LSTM ?

Les LSTM ont été conçus pour mieux conserver l'information utile sur de longues distances temporelles. Ils introduisent un **état de cellule** c_t et des **portes** contrôlant l'écriture, l'oubli et la lecture.

5.2 Équations

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(c_t). \end{aligned}$$

5.3 Rôle des portes

- **porte d'oubli** f_t : quelle part de l'ancienne mémoire garder ;
- **porte d'entrée** i_t : quelle information nouvelle écrire ;
- **candidat** $\tilde{c}_t$: contenu potentiel à stocker ;
- **porte de sortie** o_t : quelle part de la cellule exposer dans l'état caché.

La grande force du LSTM est d'offrir un chemin plus stable pour l'information via c_t . Cette structure réduit l'évanouissement du gradient et améliore l'apprentissage de dépendances longues.

5.4 Code concis

```
lstm = nn.LSTM(input_size=32, hidden_size=64, batch_first=True)
X = torch.randn(8, 15, 32)
Y, (Hn, Cn) = lstm(X)
print(Y.shape)      # (8, 15, 64)
print(Hn.shape)     # (1, 8, 64)
print(Cn.shape)     # (1, 8, 64)
```

6 GRU : Gated Recurrent Unit

6.1 Principe

Le GRU simplifie le LSTM. Il n'a pas d'état de cellule séparé; la mémoire est portée directement par l'état caché. Il repose sur deux portes principales.

$$\begin{aligned}
 z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z), \\
 r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r), \\
 \tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \\
 h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t.
 \end{aligned}$$

6.2 Interprétation

- z_t : porte de mise à jour, elle arbitre entre mémoire ancienne et information nouvelle;
- r_t : porte de réinitialisation, elle décide combien du passé utiliser pour construire le nouvel état candidat.

6.3 Comparaison LSTM / GRU

Critère	LSTM	GRU
Structure	Plus riche, 3 portes + cellule	Plus simple, 2 portes
Mémoire explicite	Oui, via c_t	Non, mémoire intégrée à h_t
Coût	Plus élevé	Plus léger
Usage	Longues dépendances complexes	Bon compromis précision/vitesse

```
gru = nn.GRU(input_size=32, hidden_size=64, batch_first=True)
X = torch.randn(8, 12, 32)
Y, Hn = gru(X)
```

7 Variantes architecturales : RNN profonds et bidirectionnels

7.1 RNN profonds

Dans un RNN profond, plusieurs couches récurrentes sont empilées. À chaque instant t :

$$h_t^{(1)} = f^{(1)}(x_t, h_{t-1}^{(1)}),$$

$$h_t^{(\ell)} = f^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}), \quad \ell \geq 2.$$

Intuition : la couche basse capture des motifs locaux, la couche haute extrait des représentations plus abstraites.

7.2 Dropout récurrent

Pour limiter le surapprentissage, on ajoute parfois du dropout entre couches. Il faut toutefois rester prudent : un dropout mal placé peut perturber la mémoire temporelle.

7.3 RNN bidirectionnels

Un RNN bidirectionnel traite la séquence dans les deux sens :

$$\vec{h}_t = f(x_t, \vec{h}_{t-1}), \quad \overleftarrow{h}_t = g(x_t, \overleftarrow{h}_{t+1}).$$

Puis on concatène :

$$h_t = [\vec{h}_t; \overleftarrow{h}_t].$$

Pour l'étiquetage morpho-syntaxique ou la reconnaissance d'entités nommées, connaître à la fois le passé et le futur d'un mot est très utile. En revanche, en génération auto-régressive, regarder le futur est impossible pendant l'inférence.

8 Préparation des données pour la traduction automatique

8.1 Pipeline général

Avant d'entraîner un modèle de traduction, on prépare un corpus parallèle source-cible :

- 1) nettoyage du texte ;
- 2) segmentation en phrases ;
- 3) tokenisation ;
- 4) construction du vocabulaire ;
- 5) ajout de tokens spéciaux ;
- 6) padding et troncature ;
- 7) création de mini-lots.

8.2 Tokens spéciaux

On rencontre souvent :

- `<pad>` : remplissage ;
- `<bos>` : début de séquence ;

- `<eos>` : fin de séquence ;
- `<unk>` : mot inconnu.

8.3 Padding et masquage

Les phrases d'un même mini-lot n'ont pas la même longueur. On les aligne avec `<pad>`. Pour éviter que ces positions artificielles influencent l'apprentissage, on utilise un masque.

$$\mathcal{L}_{\text{masquée}} = \frac{\sum_t m_t \ell_t}{\sum_t m_t},$$

avec $m_t = 1$ pour une position valide et $m_t = 0$ pour un padding.

```
def truncate_pad(line, num_steps, padding_token):
    if len(line) > num_steps:
        return line[:num_steps]
    return line + [padding_token] * (num_steps - len(line))
```

8.4 Tenseurs et mini-lots

En pratique, on obtient souvent une représentation de la forme :

$$X \in \mathbb{N}^{B \times T_s}, \quad Y \in \mathbb{N}^{B \times T_t},$$

où B est la taille du lot, T_s la longueur source et T_t la longueur cible.

9 Architecture encodeur–décodeur

9.1 Idée générale

Le cadre encodeur–décodeur sépare le traitement en deux modules :

- **encodeur** : transforme la séquence source en représentation interne ;
- **décodeur** : génère la séquence cible à partir de cette représentation.

Cette architecture est générale. Elle ne sert pas seulement à la traduction : on la retrouve en résumé automatique, dialogue, correction grammaticale, génération conditionnelle, etc.

9.2 Formalisation

Si la source est $x_{1:T_x}$ et la cible $y_{1:T_y}$, on cherche à modéliser :

$$P(y_{1:T_y} \mid x_{1:T_x}) = \prod_{t=1}^{T_y} P(y_t \mid y_{<t}, x_{1:T_x}).$$

L'encodeur produit un contexte c ou une famille de représentations. Le décodeur conditionne sa génération sur ce contexte.

9.3 Interfaces conceptuelles

- `Encoder(X)` → représentations cachées ;
- `Decoder(Y, state)` → sorties + nouvel état ;
- `EncoderDecoder(X, Y)` : pipeline global.

10 Seq2Seq récurrent

10.1 Encodeur GRU

L'encodeur lit la phrase source et renvoie une séquence d'états cachés, ainsi qu'un état final utilisé pour initialiser le décodeur.

10.2 Décodeur conditionnel

À l'instant t , le décodeur reçoit le token précédent y_{t-1} , son état caché précédent et une information de contexte issue de l'encodeur.

Une écriture simplifiée est :

$$s_t = g(y_{t-1}, s_{t-1}, c), \quad \hat{y}_t = \text{softmax}(W s_t + b).$$

10.3 Teacher forcing

Pendant l'entraînement, on fournit souvent au décodeur le vrai token précédent plutôt que sa propre prédiction. Cela stabilise l'apprentissage.

Le teacher forcing crée un décalage entre entraînement et inférence : à l'entraînement, le modèle voit la vérité précédente ; à l'inférence, il doit vivre avec ses propres erreurs passées.

10.4 Fonction de perte

On somme les pertes token par token, en ignorant les positions de padding :

$$\mathcal{L} = - \sum_{t=1}^{T_y} m_t \log P(y_t^* | y_{<t}, x).$$

10.5 Inférence auto-régressive

Au test, on commence par `<bos>`, puis on réinjecte la prédiction du modèle jusqu'à `<eos>` ou jusqu'à une longueur maximale.

```
for t in range(max_len):
    y_hat, state = decoder(dec_input, state)
    pred = y_hat.argmax(dim=-1)
    outputs.append(pred)
    dec_input = pred
```

10.6 Évaluation par BLEU

Le score BLEU mesure la qualité de la traduction générée via la précision des n-grammes, pondérée par une pénalité de brièveté. C'est une métrique standard en traduction automatique, même si elle ne capture pas toute la qualité sémantique.

11 Décodage : glouton, exhaustif et beam search

11.1 Pourquoi le décodage est difficile ?

À chaque pas, plusieurs mots sont possibles. Le nombre total de phrases candidates croît exponentiellement. Une recherche exhaustive est donc impraticable.

11.2 Décodage glouton

Le décodage glouton choisit à chaque pas le token le plus probable :

$$y_t = \arg \max_y P(y \mid y_{<t}, x).$$

Il est rapide, mais pas optimal globalement.

11.3 Score d'une séquence

Le score probabiliste d'une phrase candidate $y_{1:T}$ est :

$$\log P(y_{1:T} \mid x) = \sum_{t=1}^T \log P(y_t \mid y_{<t}, x).$$

On travaille en log-probabilités pour éviter les sous-flux numériques et transformer le produit en somme.

11.4 Beam search

Le beam search conserve les k meilleures hypothèses partielles à chaque étape. On parle de **largeur de faisceau** k .

Algorithme schématique :

- 1) initialiser le faisceau avec **<bos>** ;
- 2) développer chaque hypothèse par tous les tokens possibles ;
- 3) recalculer les scores ;
- 4) conserver les k meilleures hypothèses ;
- 5) arrêter quand les hypothèses sont terminées ou quand la longueur max est atteinte.

11.5 Pénalisation de longueur

Sans correction, le beam search favorise souvent les séquences courtes. On utilise alors parfois un score normalisé, par exemple :

$$\text{score}(y) = \frac{1}{T^\alpha} \sum_{t=1}^T \log P(y_t \mid y_{<t}, x),$$

avec $\alpha \in [0, 1]$.

Avec un faisceau $k = 3$, on garde à chaque étape les 3 meilleures suites partielles. Cela coûte plus cher que le glouton, mais donne souvent de meilleures traductions.

12 Comparaison globale des modèles

Modèle	Atouts	Limites
RNN simple	Léger, conceptuellement simple, utile pour comprendre la récursivité temporelle	Difficultés sur longues dépendances, gradients instables
LSTM	Très bon contrôle de la mémoire, robuste sur longues séquences	Plus coûteux, plus de paramètres
GRU	Plus simple que LSTM, souvent performant en pratique	Moins expressif dans certains cas
RNN profond	Représentations hiérarchiques plus riches	Coût supérieur, entraînement plus délicat
Bi-RNN	Utilise passé et futur simultanément	Inadapté au décodage causal en génération
Seq2Seq	Modélise proprement source et cible	Compression du contexte parfois difficile sans mécanismes additionnels
Beam search	Meilleure exploration que le glouton	Plus lent, sensible au réglage de k et à la longueur

13 Mini-résumé opérationnel

À retenir absolument :

- 1) Un modèle de langage factorise la probabilité d'une séquence en prédictions du prochain token.
- 2) Un RNN introduit une mémoire cachée transmise de pas en pas.
- 3) Le BPTT entraîne le RNN, mais souffre de gradients évanescents ou explosifs.
- 4) Les LSTM et les GRU stabilisent l'apprentissage grâce à des portes.
- 5) Les versions profondes et bidirectionnelles enrichissent les représentations.
- 6) Pour la traduction, il faut un pipeline de données propre : tokenisation, vocabulaire, padding, masquage.
- 7) L'architecture encodeur-décodeur modélise la traduction conditionnelle.
- 8) Le Seq2Seq s'entraîne souvent avec teacher forcing et s'évalue par BLEU.
- 9) Le beam search améliore l'inférence en explorant plusieurs hypothèses à la fois.

14 Exemple de squelette d'entraînement Seq2Seq en PyTorch

```
import torch
from torch import nn

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, src, tgt_in):
        enc_outputs, enc_state = self.encoder(src)
        dec_outputs, _ = self.decoder(tgt_in, enc_state, enc_outputs)
        return dec_outputs

# perte masquee + clipping
logits = model(src, tgt_in)
loss = masked_cross_entropy(logits, tgt_out, valid_len)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
optimizer.zero_grad()
```

15 Conclusion

Cette synthèse met en évidence une continuité pédagogique forte. On part de la **modélisation probabiliste d'une séquence**, puis on introduit des architectures capables de **mémoriser le contexte**, on apprend à les **entraîner correctement**, on les spécialise pour la **traduction automatique**, puis on affine l'**inférence** grâce au beam search.

Autrement dit :

modèle de langage → RNN → BPTT → LSTM/GRU → encodeur-décodeur →
Seq2Seq → beam search

Cette chaîne constitue une base solide pour comprendre les architectures séquentielles classiques avant de passer à des approches plus modernes comme l'attention et les Transformers.